



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 2 – Overloading and Inheritance
Topic – 2.1 Operator Overloading

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215



Introduction to Operator Overloading

- It gives additional meaning to the operator without changing its basic meaning.
- We can create our own language by using the concept of operator overloading appropriately.
- Example: + operator can be used to concatenate two strings. Also it can be overloaded to perform addition of two complex numbers.



Operator Overloading Restrictions

- Following C++ Operators can't be overloaded:
 1. Class member access operators(.)
 2. Scope Resolution Operator(::)
 3. Size of Operator(sizeof())
 4. Conditional Operator(? :)

- Precedence of operators cannot be changed
- No new operators can be created, only existing operators can assign a new meaning



General Syntax and Example

- Syntax:

Return-type operator op(arguments)

{

body;

}

- Example

void operator +()

{

Body;

}



Overloading Unary Operator

- Unary operator are those operator which works only on the single operand. Eg. ++, --, - etc
- Unary operator acts on only one operand and can be overloaded in two ways:
 1. Using member function with no arguments
 2. Using friend function with one argument where the argument must be either an object of the class or an reference to an object of the class



Unary operator overloading using member function

```
#include<iostream>
using namespace std;
class increment
{
    int m,n;
public:
    increment(int x, int y)
    {
        m=x;
        n=y;
    }
}
```

```
void display()
{
    cout<<"m= "<<m<<"n="<<n;
}
void operator ++()
{
    m=m+1;
    n=n+1;
}
};
```

```
int main()
{
    increment in1(20,30);
    in1.display();
    ++in1;
    in1.display();
    increment in2(1,2);
    in2.display();
    in2.operator ++();
    in2.display();
    return 0;
}
```



OUTPUT

```
m= 20 n=30  
m= 21 n=31  
m= 1 n=2  
m= 2 n=3  
|
```



Unary operator overloading using Friend Function

```
#include<iostream>
using namespace std;
class increment
{
    int m,n;
public:
    increment(int x, int y)
    {
        m=x; n=y;
    }
    void display()
    {
        cout<<"m= "<<m<<"n="<<n;
    }
    friend void operator ++(increment &);
};
```

```
void operator ++(increment& x)
{
    ++x.m;
    ++x.n;
}

int main()
{
    increment in1(20,30);
    in1.display();
    operator ++(in1);
    in1.display();
    increment in2(1,2);
    in2.display();
    operator ++(in2);
    in2.display();
    return 0;
}
```

Output:
m=20 n=30
m=21 n=31
m=1 n=2
m=2 n=3



Overloading Binary Operator

- Binary operators are those operators which work on two operands. Eg. +, -, *, / etc
- Binary operators can be overloaded in two ways:
 1. Using member function with single argument.
 2. Using friend function with two arguments.



Binary operator overloading using member function

```
#include<iostream>
using namespace std;
class add
{
    int m,n;
    public:
    add(int x, int y)
    {
        m=x; n=y;
    }
}
```

```
void display()
{
    cout<<"m= "<<m<<" n="<<n;
}
void operator +(add &x);
};
void add::operator +(add &x)
{
    m=m+x.m;
    n=n+x.n;
}
```

```
int main()
{
    add obj1(20,30);
    add obj2(2,3);
    obj1.display();
    obj2.display();
    obj1+obj2;
    obj1.display();
    return 0;
}
```



Output

```
m= 20 n=30  
m= 2 n=3  
m= 22 n=33  
-
```



Binary operator overloading using friend function

```
#include<iostream>
using namespace std;
class add
{
    int m,n;
    public:
    add(int x, int y)
    {
        m=x; n=y;
    }
}
```

```
void display()
{
    cout<<"m= "<<m<<" n="<<n;
}
friend void operator +(add &x,add &y);
};
void operator +(add &x, add &y)
{
    x.m=x.m+y.m;
    x.n=x.n+y.n;
}
```

```
int main()
{
    add obj1(20,30);
    add obj2(2,3);
    obj1.display();
    obj2.display();
    obj1+obj2;
    obj1.display();
    return 0;
}
```



Output

```
m= 20 n=30  
m= 2 n=3  
m= 22 n=33  
-
```

Thank You..